

When Invisible Differences Stay Invisible: Machine-Certified All-Order Congruence for Temporal Reconstruction in a Formal Navier–Stokes Architecture

Michael Zot

Independent Researcher

Corresponding author: mike@ZotBot.Ai

Public proof package: research/navier-stokes-spc/temporal-closure

Abstract

Structural Path Compression (SPC) asks whether a proof can carry distinctions that are necessary for one representation of the argument but unnecessary for the future mathematics the proof must preserve. This note machine-certifies one precise test of that idea inside the 2026 OpenAI Navier–Stokes formalization.

The baseline library already proves a one-state filtration estimate for the temporal mean reconstruction: residual inputs in `MeanClass  $\alpha$` produce a radial increment in `MeanClass ( $\alpha + 1$ )` and angular/axial increments in `MeanClass  $\alpha$` . The new contribution is not a stronger analytic estimate. It is the two-state congruence needed for compression: if two admissible states share the same reconstruction geometry and their relevant residual-source differences lie in `MeanClass  $\alpha$` , then the differences of their temporal reconstructions obey the same grade transfer. Quantifying this statement over every real order A yields an all-order theorem: inputs invisible at every finite mean-filtration order remain invisible after temporal reconstruction.

The exact 598-line Lean module compiles against `openai/NavierStokesAndEuler` at commit `f9e8bc5b38b6...` using Lean 4.34.0-rc2. The certification run returned `TRUE_LAKE_EXIT_CODE=0`; the source contains no `sorry`, `admit`, or custom axiom; and `#print axioms` for both target declarations reports only `propext`, `Classical.choice`, and `Quot.sound`. A fresh public GitHub Actions rerun has also rebuilt the published source against the same pinned baseline and returned `TRUE_LAKE_EXIT_CODE=0`; its machine-generated build log and axiom output are committed under `certification/ci/`. The complete source, checksum, reproduction script, source crosswalk, and certification evidence are published with this note.

The result closes one operator-level obligation in a larger proposed quotient-by-all-order-invisible-differences program. It does not establish full-cycle quotient closure, debt or rank closure, the full stage-free SPC theorem, or a new Navier–Stokes blowup theorem.

Keywords. Navier–Stokes; Structural Path Compression; Lean; formal verification; congruence; filtration; quotient dynamics; proof architecture.

MSC 2020. 35Q30; 03B35; 68V15; 68T15.

Baseline formalization. `openai/NavierStokesAndEuler`, commit `f9e8bc5b38b6...`

Public artifacts. research/navier-stokes-spc/temporal-closure.

The result in one page

Eureka. Compression is not proved by deleting information. It is proved by showing that the future stays blind to what was deleted. If two states differ only by something no finite accuracy level can see, then every operator used after the compression must fail to resurrect that difference.

Science underneath. For a fixed moving-strip datum s , define all-order flatness by

$$\text{Flat}_s(f) \iff \forall A \in \mathbb{R}, \text{MeanClass}_s^A(f).$$

The certified theorem concerns the temporal reconstruction operator $\mathcal{T}$. If the theta- and axial-residual source differences are in $\text{MeanClass}_s^\alpha$, then

$$\Delta\mathcal{T}_r \in \text{MeanClass}_s^{\alpha+1}, \quad \Delta\mathcal{T}_\theta \in \text{MeanClass}_s^\alpha, \quad \Delta\mathcal{T}_z \in \text{MeanClass}_s^\alpha.$$

Therefore

$$\text{Flat}_s(\Delta\theta) \wedge \text{Flat}_s(\Delta z) \implies \text{Flat}_s(\Delta\mathcal{T}_r) \wedge \text{Flat}_s(\Delta\mathcal{T}_\theta) \wedge \text{Flat}_s(\Delta\mathcal{T}_z).$$

The radial component is obtained one order better and then lowered from $A + 1$ to A using the baseline monotonicity theorem `mono_exponent`.

Eureka. The baseline theorem says, “one input of grade A produces one output of grade A .” Compression needs a different statement: “two inputs that are indistinguishable to grade A remain indistinguishable after the operator.” That change from a one-state estimate to a two-state congruence is the bridge from ordinary bounds to quotient mathematics.

Science underneath. The OpenAI source already contains `temporalIncrementState_classes`. This note adds and certifies

`NavierStokes.VariableGaugeMean.temporalIncrementState_difference_classes`

and its all-order corollary

`NavierStokes.VariableGaugeMean.temporalIncrementState_flat_classes`.

No claim is made that the $\alpha \mapsto (\alpha + 1, \alpha, \alpha)$ analytic gain is new. The formal increment is the congruence theorem and its all-order closure statement.

Eureka. A quotient becomes real one operator at a time. Every certified congruence removes one place where “discarded” information could have leaked back into the future.

Science underneath. This paper closes only the temporal-reconstruction arrow. The larger target remains

$$F(\mathcal{I}_\infty) \subseteq \mathcal{I}_\infty,$$

for the complete reduced correction cycle. Debt and later cycle components remain separate obligations.

Contents

The result in one page	2
1 The compression question becomes a congruence question	4
2 What the OpenAI formalization already proves	4

3	The new machine-certified statements	5
4	Why the formal proof is more than writing “linearity”	5
4.1	Fourier inverse commutes with subtraction	6
4.2	Centered temporal inversion commutes with subtraction	6
4.3	The radial stream potential commutes with subtraction	6
4.4	The moving-fiber bridge	6
4.5	Reuse the existing filtration estimates	6
5	The structural meaning: a local quotient certificate	6
6	Machine certification and public reproducibility	6
7	What changed during compilation	8
8	What this proves, and what it does not	8
9	Why this is useful even though the estimate already existed	8
10	Relation to Structural Path Compression	9
11	Next live theorem: debt closure	9
12	Reproducibility map	9
13	AI assistance disclosure	10
14	Conclusion	10

1. The compression question becomes a congruence question

Eureka. A long proof can confuse “how we got here” with “what the future still needs.” SPC turns that philosophical suspicion into a falsifiable condition: erase a distinction, then ask every future consumer whether it notices.

Science underneath. Let C_1, C_2 be two admissible states and let Q retain only a proposed compressed representation. To make Q a valid state reduction for an update map F , one needs a congruence implication

$$Q(C_1) = Q(C_2) \implies Q(F(C_1)) = Q(F(C_2)),$$

or the corresponding asymptotic/filtered equivalence. The present campaign uses the latter. Two states are identified when their difference lies at every finite requested order. The formal risk is that a downstream map could amplify or expose an all-order-invisible difference at some fixed finite order. A valid quotient requires ruling that out operator by operator.

The temporal mean reconstruction is one such operator. It consumes the residual sources that remain after earlier correction stages and produces a triple of mean increments. Before this work, the source library certified how one admissible source propagates through the temporal reconstruction. The missing object for the quotient program was a theorem about *differences of two states* under the same reconstruction geometry.

2. What the OpenAI formalization already proves

Eureka. The new theorem did not need a new estimate. The estimate was already there. What was missing was the right question.

Science underneath. At the pinned commit, `NavierStokes/WeightedClasses.lean` defines

$$\text{MeanClass}(s, \alpha, f) := \text{MemClass}(s, (n, x) \mapsto s.\zeta(x), \alpha, f),$$

and proves monotonicity in the exponent:

$$\text{MemClass}_\alpha(f) \wedge \beta \leq \alpha \implies \text{MemClass}_\beta(f).$$

The theorem is exposed as `mono_exponent`.

In `NavierStokes/VariableGaugeMean.lean`, the baseline library defines `temporalIncrementState`; its angular component is the temporal inverse applied to the theta residual, while its radial and axial components are reconstructed through the stream potential built from the axial residual. The same file contains `temporalIncrementState_classes`, whose relevant conclusion is schematically

$$\text{MeanClass}_s^\alpha(\theta\text{-residual}), \text{MeanClass}_s^\alpha(z\text{-residual}) \implies \begin{cases} \text{MeanClass}_s^{\alpha+1}(M_{T,r}), \\ \text{MeanClass}_s^\alpha(M_{T,\theta}), \\ \text{MeanClass}_s^\alpha(M_{T,z}). \end{cases}$$

That theorem is the analytic engine reused here.

Source-level distinction. The finite-grade exponents are not claimed as a newly discovered analytic mechanism. The new certified layer proves that the same operator is compatible with subtraction of two admissible states sharing the same fixed reconstruction parameters, and then packages that compatibility into an all-order congruence theorem.

3. The new machine-certified statements

Eureka. “Small” is not enough for a quotient. You need “small at every order,” and then you need the operator to preserve that invisibility forever. The all-order theorem is the first half of that sentence made executable.

Science underneath. Fix the same slow region, radial geometry, moving-strip parameters, temporal index map, axial direction, context, and admissibility data for two states u_1, u_2 . Write

$$\Delta\theta_n(z) = u_1.\text{thetaResidual}(c, n, z) - u_2.\text{thetaResidual}(c, n, z),$$

$$\Delta z_n(z) = u_1.\text{axialResidual}(c, n, z) - u_2.\text{axialResidual}(c, n, z).$$

Theorem 3.1 (Certified finite-grade congruence). *If*

$$\text{MeanClass}_s^\alpha(\Delta\theta), \quad \text{MeanClass}_s^\alpha(\Delta z),$$

then the componentwise temporal-increment differences satisfy

$$\text{MeanClass}_s^{\alpha+1}(\Delta M_{T,r}), \quad \text{MeanClass}_s^\alpha(\Delta M_{T,\theta}), \quad \text{MeanClass}_s^\alpha(\Delta M_{T,z}).$$

This is the compiled Lean declaration `NavierStokes.VariableGaugeMean.temporalIncrementStateDifferenceClasses`.

Corollary 3.2 (Certified all-order temporal closure). *If*

$$\forall A \in \mathbb{R}, \quad \text{MeanClass}_s^A(\Delta\theta) \text{ and } \text{MeanClass}_s^A(\Delta z),$$

then

$$\forall A \in \mathbb{R}, \quad \text{MeanClass}_s^A(\Delta M_{T,r}) \wedge \text{MeanClass}_s^A(\Delta M_{T,\theta}) \wedge \text{MeanClass}_s^A(\Delta M_{T,z}).$$

This is the compiled Lean declaration `NavierStokes.VariableGaugeMean.temporalIncrementStateFlatClasses`.

The corollary applies [theorem 3.1](#) at the arbitrary requested order A . Angular and axial outputs are already in order A . The radial output is in order $A + 1$, and the proof concludes with

$$\text{hR.mono_exponent (by linarith)}$$

to downgrade $A + 1$ to A . This is why the theorem is genuinely all-order and not the earlier, insufficient $\alpha = 0$ specialization encountered during development.

4. Why the formal proof is more than writing “linearity”

Eureka. The algebra is simple; the geometry is where a false shortcut could hide. “Subtract the two formulas” only works if both formulas really live on the same moving domain with the same support and reconstruction parameters.

Science underneath. The proof is intentionally conservative. It does not invent a new temporal inverse or new class estimate. It transports subtraction through the exact machinery already used by the baseline theorem. The formal proof has five layers.

4.1. Fourier inverse commutes with subtraction

The helper `familyInverse_sub` expands the family Fourier coefficient of $f - g$, proves coefficient subtraction, and transports it through the directional inverse series using summability and `tsum_sub`. This certifies the linearity used by the temporal inverse rather than treating it as an informal algebraic rewrite.

4.2. Centered temporal inversion commutes with subtraction

The helper `temporalAtIndex_sub` combines torus-average subtraction, centering, the source-to-family map, and the previous Fourier inverse identity to obtain exact componentwise subtraction for the temporal inverse.

4.3. The radial stream potential commutes with subtraction

The helper `streamPotential_sub` proves subtraction through the normalized weighted source and the compact/past integral representation. The proof carries the support and integrability obligations required by the source API.

4.4. The moving-fiber bridge

The moving geometry is the important proof-engineering boundary. The proof freezes the slow variable on the relevant fiber, applies the fixed-fiber subtraction identities, and returns to the moving domain using the existing fiber-locality theorems. Positivity of $qLength$ is proved at the actual moving point used by the local argument. This is the part that prevents a superficially linear argument from silently comparing objects on mismatched domains.

4.5. Reuse the existing filtration estimates

Once the difference source is established as admissible, the proof calls the existing source theorems

- `meanClass_scaledTemporalStreamBeta`,
- `meanClass_temporalAtIndex_moving`,
- `meanClass_temporalStreamGamma`,

on the difference source. The new theorem therefore inherits the source library’s analytic estimates rather than reproving them.

5. The structural meaning: a local quotient certificate

Eureka. A compression is safe when erased information has nowhere to come back from. This theorem closes one return tunnel.

Science underneath. Let

$$\mathcal{I}_\infty(s) = \{f : \forall A \in \mathbb{R}, \text{MeanClass}_s^A(f)\}.$$

For the temporal residual interface, [corollary 3.2](#) gives the operator-level preservation law

$$(\Delta\theta, \Delta z) \in \mathcal{I}_\infty(s) \times \mathcal{I}_\infty(s) \implies \Delta\mathcal{T} \in \mathcal{I}_\infty(s)^3.$$

This is precisely the kind of congruence needed if the eventual reduced dynamics are to descend through an all-order equivalence relation.

It does *not* follow from this theorem alone that the full correction map respects $\mathcal{I}_\infty$. The remaining mean-debt, rank, residual-return, and full-cycle arrows must be certified separately. The point is narrower: temporal reconstruction is no longer a live obstruction to this quotient program under the theorem’s hypotheses.

6. Machine certification and public reproducibility

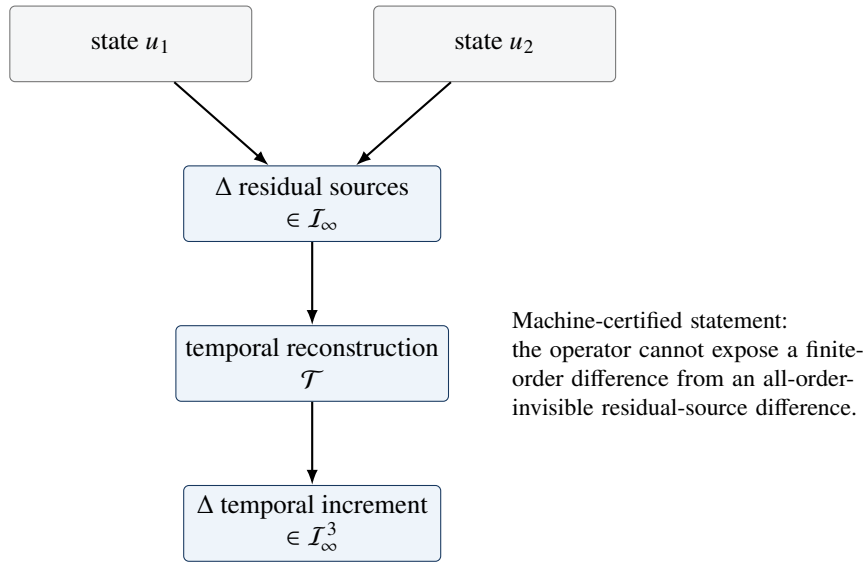


Figure 1. The certified local congruence. The result is a statement about two-state differences under shared reconstruction geometry, not a new one-state gain estimate.

Eureka. “Lean proved it” should not be a sentence readers are asked to trust. The proof object, the exact baseline, the checksum, and the commands should be public enough that the sentence can be replaced by a rebuild.

Science underneath. The certification target is `NavierStokes/PandoraTemporalFlat.lean`, 598 lines. Its SHA-256 is

13a6350607d73d101f107810986fcefb73f5b41f308be8bf78c878f43e46c95.

It was checked against `openai/NavierStokesAndEuler` at the pinned commit

f9e8bc5b38b6e212696e8a30e3e91517af887bbd

with Lean 4.34.0-rc2. The recorded successful command was

```
set -o pipefail
lake build NavierStokes.PandoraTemporalFlat 2>&1 | tee /tmp/pandora-build.log
status=${PIPESTATUS[0]}
echo "TRUE_LAKE_EXIT_CODE=$status"
```

and the recorded result was

```
Build completed successfully (3243 jobs).
TRUE_LAKE_EXIT_CODE=0
```

The 3243 figure is the Lake job count of the targeted build; the project-side import closure of the target module comprised 157 NavierStokes modules. It is not the number of files in the full repository.

The final source audit found no `sorry`, `admit`, or custom axiom. `#print axioms` for both target declarations reported only

```
{propext, Classical.choice, Quot.sound}.
```

The fresh public CI reproduction independently rebuilt this same published module against the same pinned OpenAI commit and also exited successfully. Its machine-generated build log, axiom output, exit-code record, and PASS status are committed under `certification/ci/`.

Public package. The companion repository contains the exact Lean source, SHA-256 manifest, recorded

certification output, axiom audit, source crosswalk, theorem-delta note, recorded verification-session evidence, reproduction script, CI verifier, LaTeX source, and compiled PDF. The public CI verifier has now completed a fresh successful rerun and committed its own build log, axiom output, exit code, and PASS status back into the repository, so the reproducibility layer no longer stops at a prose record.

7. What changed during compilation

The successful proof required four compiler-driven repairs to the supplied source. None weakened a theorem statement or changed the mathematical argument.

1. Open the Topology scope so the neighborhood notation $\mathcal{N}$ / `nhds` notation elaborates.
2. In the first moving-fiber use of `temporalPotential_sub_on`, prove `qLength` positivity at the local point p , not at the outer point z .
3. Make the identical moving-point repair at the second occurrence.
4. Replace a syntactically failing direct rewrite by an explicit pointwise equality `hval` derived from the same eventual-equality fact, after beta reduction.

The remaining compiler messages were non-fatal deprecation/linter warnings.

8. What this proves, and what it does not

Eureka. The exciting statement is not “we shortened Navier–Stokes.” It is smaller and sharper: one piece of the proof can now be shown, mechanically, to forget an entire class of distinctions without letting them leak back into the future.

Science underneath. The certified claim is exactly the finite-grade difference theorem plus the all-order temporal closure corollary. It establishes a local congruence property for temporal reconstruction under shared geometry and the stated admissibility hypotheses.

The following are *not* established by this note:

- full reduced-cycle preservation of $\mathcal{I}_\infty$;
- debt closure or rank closure;
- existence of a complete quotient map $\bar{F} : C/\mathcal{I}_\infty \rightarrow C/\mathcal{I}_\infty$;
- the full stage-free SPC theorem from the companion manuscript;
- a new Navier–Stokes blowup theorem;
- absolute worldwide priority for the underlying functional-analytic fact.

These boundaries are part of the claim, not caveats added after the fact.

9. Why this is useful even though the estimate already existed

Eureka. A bound tells you an object survives a calculation. A congruence tells you a distinction can be erased before the calculation. Those are different currencies of understanding.

Science underneath. The source theorem `temporalIncrementState_classes` answers a one-state question: given a source with a class bound, what class bound does its reconstruction satisfy? The new theorem answers a representation question: if two states differ only inside a specified negligible class, does the reconstruction distinguish them? Quotient constructions require the second form.

This distinction matters for proof architecture. If every consumer of a coordinate respects an equivalence relation, that coordinate need not remain independently represented merely because it appeared during one successful derivation. The mathematics can then be reorganized around equivalence classes or smaller interfaces.

Conversely, failure of any one congruence theorem produces a precise witness that the proposed deletion was invalid. The same test therefore supports both compression and falsification.

10. Relation to Structural Path Compression

Eureka. SPC began with a suspicious staircase: repeated stages looked fundamental because the proof walked them one by one. The deeper question is now sharper. Which distinctions are allowed to survive from one floor to the next?

Science underneath. The companion SPC manuscript proposes replacing part of the repeated Section 9 chronology by an accuracy-indexed compiler once positive return estimates and source legality are established. The present theorem addresses a different but complementary issue that emerged from the later causal-state audit: whether candidate equivalence classes are invariant under the actual correction operators.

Thus Paper 1 and the present note should be read as two layers:

Paper 1: recover a smaller generator from the successful path

Paper 2: certify that one deleted distinction stays deleted under a real operator

The first is primarily architectural. The second is a machine-checked local preservation result.

11. Next live theorem: debt closure

Eureka. Once one return tunnel is sealed, the frontier moves. The next question is not whether temporal reconstruction is safe; Lean has answered that. The next possible leak is debt.

Science underneath. The next targeted implication in the all-order program is schematically

$$\text{Flat}(\Delta M) \wedge \text{Flat}(\Delta \text{Cov}) \wedge \text{Flat}(\Delta M_T) \implies \text{Flat}(\Delta \text{Debt}).$$

The source file `NavierStokes/GaugeDebtIncrement.lean` contains the relevant debt-update machinery. If debt closure and the remaining rank/cycle obligations are eventually certified, the stronger target becomes

$$F(\mathcal{I}_\infty) \subseteq \mathcal{I}_\infty,$$

which would permit a well-defined induced evolution on the corresponding quotient. That statement remains open in this note.

12. Reproducibility map

Eureka. Reproducibility is strongest when every sentence of the form “we checked X” has a file or command sitting next to it. The public package is organized around that rule.

Science underneath. The public package exposes the following artifacts directly:

- `certification/PandoraTemporalFlat.lean`: exact 598-line source compiled in the certification run.
- `certification/SHA256SUMS.txt`: byte-level identity checks for the proof and publication artifacts.
- `certification/CERTIFICATION_RECORD.md`: environment, command, result, theorem names, and claim boundary.
- `certification/BUILD_RESULT.txt`: recorded successful Lake command/result.

- `certification/AXIOMS.txt`: recorded axiom audit for both target declarations.
- `certification/SOURCE_MAP.md`: exact baseline definitions and theorems reused from the pinned OpenAI tree.
- `certification/THEOREM_DELTA.md`: existing one-state estimate versus the new two-state and all-order statements.
- `certification/REVIEW_RESPONSE.md`: reviewer concern-to-fix crosswalk.
- `REPRODUCE.md` and `scripts/reproduce.sh`: independent rebuild and audit procedure.
- `.github/workflows/verify-navier-stokes-temporal.yml`: public CI verifier; successful reruns publish machine-generated logs under `certification/ci/`.
- `PUBLICATION_MANIFEST.md`: full skeptical-reader artifact manifest.
- `CLAIM_BOUNDARY.md`: explicit certification boundary.

13. AI assistance disclosure

AI systems were used as research tools for source retrieval, proof-dependency analysis, formal-proof drafting and debugging, adversarial checking, literature/prior-art search, and editorial drafting. The research direction, conceptual framework, selection and interpretation of claims, acceptance or rejection of proposed arguments, and publication decisions were determined by the author. The final theorem status reported here is tied to the public Lean artifact and reproducible build procedure, not to an AI-generated prose assessment.

14. Conclusion

Eureka. The most useful thing a compression theorem can prove is not that a path is shorter. It is that the future has lost the ability to tell which path you took.

Science underneath. The OpenAI formalization already supplied the analytic temporal reconstruction estimate. This work adds the two-state congruence and all-order closure required by the emerging SPC quotient program and checks both declarations in Lean against a pinned public baseline. The theorem is deliberately local, but its role is structural: temporal reconstruction cannot turn an all-order-invisible residual-source difference into a finite-order-visible temporal increment under the certified hypotheses.

That is one closed arrow. The larger quotient theorem remains to be earned arrow by arrow.

References

- [1] OpenAI, *Finite Time Blowup for Navier–Stokes*, 2026. Formal source repository: <https://github.com/openai/NavierStokesAndEuler>. Pinned baseline used here: commit `f9e8bc5b38b6e212696e8a30e3e91517af887bbd`.
- [2] M. Zot, *Navier–Stokes: Recovering Minimal Structure from Representation-Induced Complexity*, working preprint, September 2026. <https://github.com/mikecreation/ZotBot/tree/main/research/navier-stokes-spc>.
- [3] L. de Moura and S. Ullrich, “The Lean 4 Theorem Prover and Programming Language,” in *Automated Deduction – CADE 28*, 2021.